QMX+ Panadapter — User Guide (v0.18.7)

By Steffen Lav (OZ1LAV). Generated from the project README — for the live page, issues, and releases see https://github.com/SteffenLav/qmx-panadapter

---

Contents

Chapters

Appendices

---

1. Quick Guide

Step 0 — Check your QMX firmware first

Power the QMX on by itself and read the firmware version off its own display at boot. You need **1.03.002 or newer**. If yours is older, update the QMX *before* connecting the Tab5 — everything that follows depends on it. This takes 10 seconds to verify and saves hours of debugging.

Step 1 — Flash the Tab5 firmware

Use the one-click flasher in `tools/QMX-Panadapter flasher/` (also attached to each release):

1. Plug the Tab5 into your computer with a **USB-C data cable** — charge-only cables will not work.
2. Run the flasher:
   - **Windows** — double-click `flash.bat` (downloads esptool + firmware automatically; nothing to install)
   - **macOS** — double-click `flash.command` (needs esptool once — `brew install esptool` recommended; `pip3 install esptool` often fails on recent macOS with an "externally-managed-environment" error). If macOS blocks the script, right-click → **Open**, or run `bash flash.command` in Terminal (no `chmod` needed).
   - **Linux** — `bash flash.command` (needs `pip3 install esptool`)
3. The flasher asks **normal or clean** (see below) — just press **Enter** for a normal flash.
4. Wait for `SUCCESS`. The Tab5 restarts on the new firmware.

The flasher downloads the latest release from GitHub automatically. **No internet?** Put a `qmx_panadapter_merged_*.bin` from the releases page next to the flasher and it uses that instead.

Normal vs clean flash. Just before flashing, the flasher asks:

> *Type E for a CLEAN/ERASE flash, or just Enter for a normal flash*

- **Normal (press Enter)** — updates the firmware and **keeps** all your saved settings (WiFi, callsign, grid, memory channels, ADIF log). This is what you want almost every time.

- **Clean (type E)** — wipes the whole chip first, so **every saved setting is permanently erased**: WiFi name *and* password, callsign/grid, all memory channels, and the logged QSOs. Use it only if something is stuck or corrupted (e.g. WiFi refuses to turn on no matter what). Back up first with **Config** ↓ (below) so you can restore in seconds afterwards.

Building from source? See Build from source.

Step 2 — Connect the cables

You need **two** USB connections, and the cable to the QMX is the one people get wrong:

Connection	Port on Tab5	Cable	Carries
Tab5 → QMX	**USB-A** (host)	**USB-A → USB-C, full data cable**	I/Q audio (UAC) + CAT (CDC-ACM)
Tab5 → power	**USB-C**	Any USB-C power cable (5 V / 2 A+)	Power

> **The #1 failure is a charge-only cable.** Many USB-C cables — especially thin ones bundled with phones and chargers — carry power only, no data lines. If you use one between the Tab5's USB-A port and the QMX, the Tab5 powers the QMX but sees no audio and no CAT: the spectrum stays flat and the top bar shows `Band: ---`. Use a cable you know does data (one that works for a USB stick or phone file transfer). When in doubt, swap the cable first.

Power-on order matters. Turn the **Tab5 on first** and let it finish loading, then turn the **QMX on**. Within a few seconds the top bar should populate Band / Mode / BW and the spectrum should come alive.

Once flashed you can power the Tab5 from any 5 V/2 A USB-C source or the internal battery — the laptop is only needed for flashing. For diagnostics the Tab5 also outputs a serial log over the USB-C data connection (useful if WiFi is not available), but for most users the built-in **Diagnostic log** toggle in the settings drawer is the easier path — see Step 7.

Step 3 — Find your way around

The whole app is driven by **one-finger swipes from the screen edges** and **taps on the top bar**. The settings drawer is always one swipe away — no hunting for hidden menus.

Gesture	From	Does
Swipe →	Left edge	Toggle Panadapter ↔ FT8 screen
Swipe ←	Right edge	Open settings drawer
Swipe ↑	Bottom edge	Open memory channel picker
Tap or swipe ↓	Any top-bar item	Open that item's selector

Slim "breathing" grip handles on each edge show where to swipe — faint enough not to clutter the display, visible if you look for them.

The top bar reads **Band · Mode · BW · Freq · Signal · Zoom** left to right. Tap any item to open a selector — **Freq** opens the frequency keypad; **Mode** switches USB/LSB/CW/DiGi; **BW** selects SSB filter width (USB/LSB only); **Band** jumps to a configured band; **Zoom** selects a zoom preset.

See the full gesture table in the Reference section.

Step 4 — Fill in your settings

On first boot the Tab5 opens a setup prompt automatically — it will ask for your callsign, grid square, and WiFi credentials before you reach the main screen. You can skip any field and fill it in later via the settings drawer.

To open the settings drawer at any time: swipe in from the right edge, or tap the right grip handle.

- **WiFi** — enter your SSID and password. Recommended for everyone: you get accurate UTC time (needed for FT8 slot timing), the browser web UI, and remote diagnostics. The **WiFi initiated** checkbox below the WiFi section is an on/off master switch — uncheck it for POTA or field use with no network.
- **Identity** (callsign + grid square) — *only required for FT8 transmit*. Enter your callsign and Maidenhead grid (4 or 6 characters, e.g. `JO45` or `JO45ab`). The grid also drives the distance and bearing columns in the FT8 decode list.

Everything else — dB range, smoothing, colour map, brightness, IQ balance — has sane defaults. Adjust if you want to, but you don't need to on first use.

Step 5 — Using the panadapter

Switch the QMX to any band and watch it come alive. This is the foundation of the device regardless of which modes you operate.

Tap or drag to tune. Touch anywhere on the spectrum or waterfall to place the cyan cursor. Drag and it snaps to a mode-aware grid — 10 Hz steps in CW, 250 Hz in SSB, 500 Hz for FT8 — so you can land precisely on a signal before lifting your finger. Lift, and the QMX retunes.

Swipe fast to pan instead. A quick horizontal swipe (rather than a held drag) slides the whole view left or right and retunes to wherever you let go — the fastest way to slide along a band. See Touch-to-tune for exactly how the two gestures are told apart.

Zoom in on a crowded band. Pinch with two fingers to zoom up to ×24. On a CW band at ×8 or higher you can resolve individual signals a few hundred Hz apart, read the spacing between callers, and pick your target before you tune. The frequency axis scales with you, down to Hz precision at high zoom. Double-tap anywhere to reset to the full 48 kHz view.

Find your place on the band. The thin band-plan strip under the frequency axis colour-codes the CW/Digi/Phone segments around you — pick your IARU region (or leave it on Auto) in the settings drawer → **Band-plan region**.

Read your passband. Two grey vertical lines on the spectrum mark your current filter edges. The BW label in the top bar shows the active width; tap it to choose 2.5 / 2.7 / 2.9 / 3.2 kHz in USB or LSB. A coloured tint fills the passband so you can always see exactly what slice of the band you're receiving.

Watch the S-meter. The Signal field in the top bar is a live tick-scale bar (S1 through S9+20), driven by the actual signal under your VFO cursor.

Flat-spectrum mode. Toggle **Flat spectrum** in the settings drawer to switch from absolute dBm to dB-above-local-noise-floor. Noise collapses to a calm baseline; real signals — including weak CW tones in the mud — pop sharply above it. Recommended on noisy bands.

Save your spots. Swipe up from the bottom edge to open the memory channel picker. Long-press an empty slot to save the current frequency and mode with a label. Tap to recall — the QMX retunes instantly.

Monitor from another room — or operate remotely. Once WiFi is configured, open `http://<tab5-ip>` in any browser for a full-featured live panadapter with mouse-driven tuning, band/mode controls, and ADIF log download. The IP is shown in the bottom status bar. See Web UI.

Step 6 — FT8 (if that's your thing)

Swipe in from the **left edge** to switch to the FT8 screen. The Tab5 starts decoding 15-second slots immediately — no PC, no WSJT-X required.

1. Tap the **Preset** button and pick your band's conventional FT8 dial frequency (e.g. 14.074 MHz for 20 m).
2. Watch the decode list fill. CQ stations appear at the top sorted by SNR; exchanges and replies below.
3. To reply to a station: **hold your finger on their row** for ~250 ms. A dim highlight appears after ~80 ms so you can see which row you're targeting before the gate fires. Lift — a confirmation modal shows the exact message before anything is armed.
4. Tap **Auto Pounce** to hand the full QSO to the auto-engine (works through report → RR73 → 73 with patient retry), or **Transmit** for a single manual message.
5. To call CQ: tap **Call CQ**. The engine picks a clear audio slot, fires CQ, automatically answers the first caller, runs the full exchange, logs the QSO, and resumes calling CQ.

Every completed QSO is written to an ADIF log downloadable from the web UI. See FT8 Transmit for the full picture.

Step 7 — Something not working?

Spectrum flat, top bar shows `---`:

- Check the cable between Tab5 and QMX — almost always a charge-only cable.
- Make sure the QMX is powered on and not stuck in its own bootloader.
- Power cycle in order: Tab5 first, then QMX.

For anything else:

1. Open the settings drawer → flip **Diagnostic log** ON (top row).
2. Reproduce the problem (let it run a minute; power-cycle the QMX if the issue is about connection).

3. Grab the log:
    - **Over WiFi:** browse to `http://<tab5-ip>/api/log` or click **Diag log** ↓ in the web UI bottom bar — downloads `qmx-log.txt`.
    - **Over USB (no WiFi needed):** capture the serial console with `tools/capture_serial_log.ps1`.
4. Open an issue and attach the log. It includes Tab5 and QMX firmware versions plus every CAT command exchanged — usually enough to pinpoint the problem immediately.

---

2. Panadapter

Spectrum and waterfall

The display is divided into a **60 px top bar**, a **200 px spectrum** (green curve with dim fill), a **32 px frequency axis**, a **22 px band-plan strip**, a **370 px waterfall**, and a **36 px bottom bar**. The full visible span is 48 kHz centred on the QMX VFO.

The **spectrum** shows signal power in dBm (default range −130 to −30 dBm). Each frame is smoothed per-bin with an exponential moving average (EMA α = 0.4 by default, adjustable in the drawer), balancing visual stability against snappy response to CW signals and SSB attack transients.

The **frequency axis** shows absolute MHz labels centred on the QMX VFO, refreshed on every CAT frequency update. At high zoom levels the labels resolve to kHz or Hz precision.

Band-plan strip. A thin coloured bar directly under the frequency axis shows the coarse CW / Digi / Phone segments of the current band, with a marker for where the VFO sits within them. Pick which IARU region it reflects in the settings drawer → **Band-plan region** (Auto from your grid square, or a fixed Region 1 / 2 / 3).

The **waterfall** runs newest row at the top in a thermal SDR palette (black → dark blue → teal → green → yellow → red). Four colour maps are available in the settings drawer: Thermal, Viridis, Turbo, and Grayscale.

Waterfall floor tracking. The waterfall's black level tracks the band noise automatically — a running median sampled only from bins inside the passband, EMA-smoothed — so the background colour follows conditions rather than a fixed anchor. Bins outside the passband run darker and are excluded from the floor calculation so they don't wash out dim in-band signals.

Flat-spectrum mode (toggle in the settings drawer, persisted to NVS) switches both spectrum and waterfall to a per-bin adaptive display: each bin renders as dB above its own running noise floor. Real signals — including weak CW tones — stand out sharply against a calm baseline. This is the recommended mode on noisy bands. The dB range sliders have no effect in flat mode; the axis shows relative dB above floor.

Touch-to-tune

Tap anywhere on the spectrum or waterfall to place the cyan tune cursor. Drag and the cursor snaps grid-point to grid-point — you can see exactly which frequency will be tuned before you lift. The snap grid is **mode-aware**:

Tune vs. pan — how your finger is read. A quick horizontal swipe (more than ~70 px within 250 ms) pans the view instead of tuning — see Zoom and pan. A slower touch-and-hold (250 ms without that much movement) locks into tune mode, after which dragging moves the snap cursor as described below. In short: swipe fast to pan, hold-then-drag to tune.

Mode	Snap step
CW / CW-R	10 Hz
USB / LSB	250 Hz
FT8 / DiGi / RTTY	500 Hz
AM / FM	1 kHz

The grid is anchored to absolute frequency (e.g. …200 / 300 / 400 Hz), not to your touch start point, so the cursor always lands on the same set of grid points regardless of where your finger first touches.

A floating frequency tooltip above the cursor shows the target frequency in real time while dragging. Lift → CAT `FA` command is sent; QMX retunes; spectrum re-centres.

Snap-to-strongest-bin. At zoom ×1 a tap searches ±700 Hz around the touched position for the strongest spectrum bin and snaps to it — only when the peak exceeds the local mean by >3 dB, so touches on empty noise floor still tune to the raw position. Toggle this in the settings drawer → **Snap to signal** (on by default); turn it off if you want taps to always tune to exactly where you touched.

Passband indicator. Two grey vertical lines mark your current filter edges. A faint coloured tint fills the passband. The amber VFO marker shows where the QMX is tuned; in CW mode it sits at dial + CW pitch offset so it marks the actual received tone frequency, not the suppressed carrier.

Zoom and pan

Gesture	Effect
One-finger fast horizontal swipe	Pan/"stroll" the view — retunes to the new centre frequency on release
Pinch (two fingers)	Zoom ×1.0 – ×24.0
Two-finger drag	Pan the zoomed window
Double-tap	Reset zoom and pan to ×1.0 / centred
Top-bar Zoom → tap	Zoom preset: ×1 / ×2 / ×4 / ×8 / ×16 / ×24

One-finger pan (stroll). A fast horizontal swipe — more than ~70 px of movement within the first 250 ms of touching down — slides the spectrum and waterfall under your finger in real time, with a live frequency tooltip, and retunes to wherever you release. This works at any zoom level alongside the two-finger pinch/pan above; it's the quickest way to slide along a band without lifting into a deliberate tune-drag.

At zoom > ×1 the view **centres on the passband** (not the VFO dial), which matters for USB/LSB where the passband sits offset from the carrier. The passband lines and frequency axis track correctly at all zoom levels. Zoom level is persisted to NVS and restores with full zoom-FFT resolution on the next boot.

S-meter

The Signal field in the top bar is a visual tick-scale bar labelled S1, S3, S5, S7, S9, +10, +20 with a moving green bar beneath. It is driven by the peak dBm in a ±64-bin window centred on the IF-shifted VFO bin — the actual signal under your cursor, not the DC/LO artefact at bin 0.

S-unit mapping: S9 = −73 dBm; 6 dB per S-unit below S9; 1 dB per unit above S9. The meter stays live during FT8 capture.

Memory channels

Swipe up from the bottom edge (or tap the bottom grip handle) to open the 4×8 memory channel grid (32 slots, NVS-persisted).

Action	How
Recall	Tap a slot — QMX retunes to stored frequency and mode
Save	Long-press an empty slot — frequency/mode picker opens, then a label keyboard
Edit / delete	Long-press an occupied slot

Memory slots show the label in large text and mode + frequency (dimmed) below. The frequency/mode picker pre-fills the current VFO and lets you edit both before naming.

Settings drawer

Open by swiping in from the right edge, or tapping the right grip handle. The drawer is scrollable.

Controls appear top to bottom in this order:

Control	What it does
Flip 180°	Inverts the whole display and touch axes for upside-down mounting; centred checkbox so it isn't hit by accident, persisted
Diagnostic log	Captures all firmware log output to a 512 KB ring (stays visible in FT8 mode)
IQ Balance	Toggle adaptive I/Q image correction; re-enabling resets the estimator
Flat spectrum	Toggle flat/absolute display mode, persisted
Snap to signal	Toggle snap-to-strongest-bin on tap (see Touch-to-tune); on by default
Presets	HF Normal / HF DX / Strong Sig — sets dB range and smoothing in one tap
WiFi	Opens credential modal; **WiFi initiated** checkbox enables/disables WiFi entirely
Identity	Callsign + Maidenhead grid (required for FT8 TX; also drives KM/BRG columns)
dB Range	Min and Max sliders (dBm)
Smoothing	EMA alpha 0.05–1.00
CW	CW sidetone centre, 600–800 Hz; touch-to-tune in CW mode snaps to this offset, persisted
CW Audio	*Currently disabled (greyed out).* Would play demodulated CW out the Tab5's speaker/headphone jack; shelved pending a USB-audio pipeline fix — see CW Audio below
IF calibration	±100 Hz trim for per-unit LO variance (see Per-unit IF calibration)
Display	Brightness, 10–100%, persisted

Control	What it does
Waterfall colour map	Thermal / Viridis / Turbo / Grayscale, persisted
Band-plan region	Auto (from your grid square) / Region 1 (EU/AF) / Region 2 (Americas) / Region 3 (Asia/Pac) — drives the band-plan strip
Waterfall	Black level, Contrast, Adaptive floor blend, and FFT window — see Waterfall colourisation below

Waterfall colourisation

Four live, NVS-persisted sliders/dropdown at the bottom of the settings drawer fine-tune how the waterfall maps signal to colour — changes scroll in from the top as you drag:

Control	Range	Default	Effect
Black level	0–30 dB	9 dB	How far above each bin's own noise floor a signal needs to be before it lifts off black
Contrast	10–80 dB	45 dB	The dB span that fills the rest of the colour ramp above the black level
Adaptive floor	0–100%	100%	Blend between a per-bin noise floor (100%) and one global mean floor across the band (0%)
FFT window	Blackman-Harris / Hann (sharp) / Nuttall	Blackman-Harris	The FFT window function; Hann trades some sidelobe suppression for sharper peaks

CW Audio (disabled)

A **CW Audio** section appears in the drawer — greyed out, with its checkbox forced off — for a feature that plays demodulated CW out of the Tab5's own speaker/headphone jack while in CW/CW-R mode. It's shelved for now: enabling it was found to cut FT8 decode throughput by 2–3× even when not actively playing audio, most likely USB-audio/DMA contention, and a proper fix needs a redesigned audio path. The section stays visible (with a volume slider, also disabled) as a placeholder for when that lands.

3. Web UI

With the Tab5 on WiFi, open `http://<tab5-ip>` in any modern browser. The IP is shown in the bottom status bar on the Tab5.

The browser panadapter is a full-featured view in its own right — not just a window onto the Tab5. On a larger monitor you get more spectrum history, a bigger waterfall canvas, and mouse controls that are faster than touch for precise tuning. It shows live spectrum at ≈10 fps via WebSocket, full waterfall history (~50 s), the same thermal palette and floor maths, a graphical S-meter, and a top bar with Band / Mode / BW / Zoom controls. The bottom bar shows battery percentage + voltage, firmware version, a live UTC clock, and WiFi SSID + RSSI. To its right: download/upload buttons (ADIF, QRZ, eQSL — see QSO logging), **Diag** ↓ for the diagnostic log, **Config** ↓ **/ Config** ↑ to back up / restore / edit all settings (see Config backup, restore & edit), and **Tab5Shot** which opens a live `/ss.bmp` screenshot in a new tab.

Click or drag to tune. Click or drag on the spectrum or waterfall — a cyan cursor appears with a live frequency readout and commits on release.

Mouse-wheel to pan or tune. Rolling the mouse wheel over the spectrum or waterfall **pans** the view when zoomed in, letting you survey the band without touching the Tab5. At zoom ×1 the wheel **tunes** with mode-aware snap (CW 10 Hz, SSB 500 Hz, DiGi 100 Hz, AM/FM 1 kHz).

Band / Mode / BW / Zoom dropdown pills in the top bar send commands to the QMX via `/api/cmd` — the same effect as using the Tab5 top bar.

Zoom sync. The browser renders the same zoomed window as the Tab5.

ADIF log download. Once you have logged at least one completed FT8 QSO, a **"N QSOs ↓"** link appears in the bottom bar of the web UI. Clicking it downloads your `qso.adi` file directly. The link is only shown when the log contains data — it disappears after clearing the log with `/api/adif/clear`.

QRZ / eQSL upload. Two more buttons appear alongside the ADIF link once you have logged QSOs — see QSO logging for the full picture.

Config backup, restore & edit

The **Config ↓ / Config ↑** buttons in the bottom bar let you save every setting to a plain text file, edit it on your PC, restore it, or share parts of it.

- **Config ↓** downloads `qmx-config.txt` — a human-readable INI-style file with everything the Tab5 remembers, grouped into sections:
  - `[settings]` — callsign, grid, WiFi SSID/password, CW pitch, IF trim, IQ balance, flat-spectrum, zoom, colour map, brightness, dB range, EMA, diag-log/GPS toggles, keypad layout, QRZ key, eQSL user/pass.
  - `[cq]` — the three FT8 CQ-message presets and which is active.
  - `[ft8_filters]` — the CQ-run include/exclude filter terms and toggles.
  - `[memories]` — the 32 memory channels, one per line: `slot = freq_hz, mode, label`.
- **Config ↑** picks an edited file and **merges** it back: only the keys and sections present in the file are changed — everything else is left as-is. Unknown keys are ignored (so a file from a newer firmware still loads). Memory channels apply immediately; other settings take effect after a restart.

Three things you can do with it:

1. **Back up before a clean flash.** Hit **Config ↓** first, do the clean/erase flash, then **Config ↑** to restore everything in seconds — instead of re-typing it all on glass.
2. **Edit in a text editor instead of on the touchscreen.** Faster for callsign, grid, CQ messages, or punching in a batch of memory channels.
3. **Share a band plan.** Copy just the `[memories]` section into a message — another operator pastes it into their file and uploads it to get the same channels (their other settings untouched).

> The downloaded file contains your **WiFi password** and **QRZ/eQSL credentials in clear text** (so it works as a full backup). Keep it private; strip those lines before sharing a file with anyone.

Endpoints

Endpoint	Method	Returns
`/`	GET	Browser panadapter (HTML)
`/api/status`	GET	JSON status (see below)
`/api/cmd`	POST	Send Band/Mode/BW/Zoom commands
`/api/log`	GET	Diagnostic log download (`qmx-log.txt`)
`/api/adif`	GET	ADIF QSO log download (`qso.adi`)
`/api/adif/clear`	GET	Wipe ADIF log and worked-call cache
`/api/qrz_key`	POST	Save QRZ Logbook API key (body = raw key text)
`/api/qrz_upload`	POST	Upload pending QSOs to QRZ Logbook; returns `{uploaded, failed, error}`
`/api/eqsl_creds`	POST	Save eQSL username/password (JSON body `{"user","pswd"}`)
`/api/eqsl_upload`	POST	Upload pending QSOs to eQSL (batched); returns `{uploaded, failed, error}`
`/api/config`	GET	Download all settings + memory channels as editable INI (`qmx-config.txt`)
`/api/config`	POST	Upload an INI config; merges into NVS; returns `{applied}`
`/ss.bmp`	GET	1280×720 RGB565 BMP screenshot
`/ws`	WS	Binary spectrum stream (~10 fps)

`/api/status`

```
{
  "battery":     { "level": 100, "mv": 8320, "charging": true },
  "wifi":        { "ssid": "MyNet", "rssi": -44, "ip": "192.168.1.213" },
  "freq_hz":     14074000,
  "mode":        "USB",
  "band":        "20m",
  "passband_hz": 2700,
  "signal_dbm":  -87.4,
  "zoom":        1.0,
  "pan_bins":    0,
  "flat_mode":   false,
  "utc_epoch":   1750000000,
  "qso_count":   12,
  "qrz_key_set": false,
  "eqsl_creds_set": false,
  "tab5_fw":     "v0.18.7",
  "qmx_fw":      "1_03_002QMX"
}
```

Polled at 1 Hz by the landing page; safe to consume from monitoring scripts, home automation, etc. `qmx_fw` is read from the QMX via the `VN;` CAT command at link-up (empty until the radio responds). `signal_dbm` is the peak dBm around the IF-shifted VFO bin (null if DSP has no data yet). `qrz_key_set` / `eqsl_creds_set` tell the web UI whether to prompt for credentials before the next upload.

`/ws` — binary spectrum WebSocket

Single-client endpoint. Each binary frame is 1026 bytes:

Bytes	Meaning
`0`	Frame type: `0x01` = spectrum
`1`	Zoom decimation factor (1 = base 1024-bin spectrum, >1 = zoom-FFT with that decimation)
`2–1025`	1024 unsigned bytes, one per bin, quantised −130 dBm (0) to −30 dBm (255)

Bins are pre-shifted server-side so byte 2 is the leftmost screen pixel. Push rate ≈10 fps.

Screenshots

```
Invoke-WebRequest -Uri "http://192.168.1.213/ss.bmp" -OutFile screenshot.bmp
```

---

4. FT8 Receive

Live FT8 reception on 20 m (v0.15.7). Left pane: MODE / Preset freq / UTC / slot countdown with parity and progress bar / TX parity preference / operator identity / Call CQ / decode summary. Right pane: scrollable decode list.

Swipe in from the **left edge** to switch to the FT8 screen. The Tab5 decodes 15-second FT8 slots directly on the ESP32-P4 — no PC, no WSJT-X.

Prerequisites

- **WiFi configured** — recommended for accurate UTC time. Without it the Tab5 falls back to its supercap RTC or the QMX clock. See Time sync.
- **Callsign and grid** set in the drawer → **Identity** if you want distance/bearing columns or plan to transmit.

The FT8 screen

Left pane: MODE label · **Preset** frequency button (tap to pick a band's conventional FT8 dial frequency) · UTC clock · 15-second slot countdown with current parity (EVEN blue / ODD amber) and a gliding progress bar · TX: EVEN / TX: ODD parity preference buttons · Active station count · **Call CQ** button · last-slot decode summary.

Right pane — decode list:

Column	Content
SL	Slot parity: **E** (blue, :00/:30) or **O** (amber, :15/:45)
CALL	Extracted callsign
MESSAGE	Full decoded FT8 message text
COUNTRY	DXCC entity from prefix lookup (~190 entities)
SNR	FFT-based estimate, colour-banded: green ≥0 / white −5..−1 / orange −15..−6 / grey <−15
KM	Great-circle distance from your grid
BRG	Bearing from your grid
HRD	Times decoded since last appearance

CQ calls always appear at the top sorted strongest-SNR first; all other rows follow by SNR descending.

Row colour scheme (CALL + MESSAGE columns): **white** for ordinary traffic, **green** for plain `CQ` calls, **dim grey** for callsigns already worked **on the current band** (a station you worked on 20 m still shows normally on 40 m — see Reply filter for hiding them entirely), and **inverted red fill + white text** for any message containing your own callsign — your highest-priority rows literally pop off the screen instead of relying on red-on-black text, which field testing found hard to read at a glance.

Live view. Stations not re-decoded within 60 seconds drop off the list automatically, even while the band is quiet. The count reads "Active: N" — who's on frequency *now*, not a cumulative history.

Performance

On a busy 20 m FT8 slot the decoder regularly yields 25–50 callsigns per slot. Both EVEN and ODD slots decode every cycle via a ping-pong dual-buffer architecture — a TX slot never causes the opposite parity to be skipped. Heap is stable across long sessions (≈39 KB internal RAM free, 25 MB PSRAM free during decoding).

---

5. FT8 Transmit

> **Known limitations (beta):** no duty-cycle protection (the firmware will not refuse consecutive TX slots), no audio loopback verification, multi-hour TX soak testing not yet completed. Use a dummy load for first tests; a power/SWR meter is useful but not required — the Tab5 reads `PC;`/`SW;` from the QMX after each burst and shows the result. The QMX has a built-in CAT timeout (120 s default) that returns it to RX if the Tab5 stops sending commands.

The Tab5 transmits FT8 via the QMX's `TA<freq>;` CAT command — no PC audio path, no WSJT-X. The QMX does all DDS synthesis and envelope shaping; the Tab5 sends the 79 tone-frequency commands at 160 ms cadence, bracketed by `TX;` / `TA0;` / `RX;`.

Replying to a station

1. In the decode list, **hold your finger on a row** for ~250 ms. A dim highlight appears after ~80 ms so you can see which row you're targeting. List scroll locks once the gate fires; drag up or down to land on the right row.
2. **Lift** — a confirmation modal shows the exact message that will go on air, the audio frequency, and the target slot parity.
3. Tap **Auto Pounce** to hand the full QSO to the auto-engine, or **Transmit** for a single manual message.
4. Tap **Cancel** (in the modal, or the armed indicator in the left pane) to disarm without transmitting.

The reply is the standard FT8 exchange: `<their_call> <my_call> <my_grid>`. Slot parity is set automatically — if you heard them on an EVEN slot, your reply goes on ODD so they're listening when you transmit.

The auto-engine works the full exchange: TX1 (grid) → wait for their report → TX2 (R+report) → wait for RR73/73 → TX3 (73) → DONE. At every step it re-sends the current message for up to 4 consecutive slots if the other station doesn't respond. If no reply comes after 4 slots, the QSO times out (orange status, tap to clear).

Calling CQ — CQ-run mode

Tap **Call CQ**. No confirmation modal. The engine:

1. Scans the current decode list for occupied 50 Hz audio bins.
2. Picks the nearest unoccupied bin to 1500 Hz (standard FT8 sub-band centre).
3. Arms the CQ on the matching slot parity and waits for the boundary.

From there it's fully automatic:

- The opposite-parity slot is decoded while CQ runs so replies are never missed.
- The moment a station replies with your callsign, CQing stops and the exchange starts automatically — best-SNR caller chosen if multiple stations answer simultaneously.
- After the exchange completes (RR73 → 73 → logged), CQ resumes on the same frequency.
- A mid-exchange timeout also resumes CQ rather than dropping the frequency.
- While CQ-run is active, other stations' `CQ` rows are hidden so replies to you stand out.

Tap **Cancel** in the TX status bar at any time to stop.

Reply filter

Tap **Filter** in the left pane to open the filter editor. Controls which stations CQ-run auto-answers and which rows appear in the decode list.

- **Include 1 / Include 2** — if either field has text, only messages containing one of its terms are eligible. Space- or comma-separated (e.g. `POTA SOTA` or `JA, VK`), matched against the *whole* decoded message text so POTA/SOTA tags, grid squares, country prefixes, `/P` suffixes etc. all work.
- **Exclude 1 / Exclude 2** — messages containing any of these terms are skipped even if they'd otherwise match.
- **Exclude plain CQ callers** — hides bare `CQ ...` rows so only replies and exchanges remain visible.

- **Exclude worked-before** — hides callsigns already worked **on the current band** (per-band, from your ADIF log) from the list and from CQ-run auto-replies; the same station on a different band is treated as not worked.

Tap **Save** to persist (NVS) and apply immediately.

Auto-answer CQ (robot)

⚠ **Transmits unattended — never leave it running unsupervised.** When enabled, the robot picks a CQ caller every slot (filtered the same way as above, plus a worked-before skip) and runs the full exchange itself — no per-QSO confirmation, no tap required. Pick a **Priority**: Strongest signal, Weakest signal, or Most distant grid. Same TX1→report→RR73→73→ADIF-log flow as a manually-tapped reply; you just aren't the one tapping. You remain responsible for everything it transmits under your callsign, same as any other unattended digital-mode software.

TX status indicator

The left pane shows a persistent status line below the slot countdown:

TX status indicator

The left pane shows a persistent status line below the slot countdown:

Colour	Meaning
Red	`TRANSMITTING: <message>` — burst in progress; tap to abort
Amber	`TX armed: <message> → EVEN/ODD, ~Ns` — waiting for slot; tap to cancel
Red-orange ⚠ FREQ BUSY	Armed tone is occupied (±50 Hz guard). During CQ-run this self-heals — the next cycle automatically hops to the nearest clear tone. Mid-exchange (replying to a specific station) the tone is locked to match them, so it just rides out until the exchange ends
Green	QSO complete
Orange	QSO timed out — tap to clear
Dim white	FT8 engine status passthrough (capturing / decoding / symbol count / …)

QSO override buttons

During an active auto-QSO exchange (any state between first reply and final 73), three buttons appear in the left pane:

Button	What it does
Re-send / <field> (amber)	Re-arms the current outgoing message immediately. The label shows what will go out — e.g. **Re-send / JO45** when waiting for their report, **Re-send / -07** when waiting for RR73
RR73 (blue)	Skips straight to RR73, bypassing the report step
73 (green)	Fires the 73 sign-off and closes the QSO immediately

These are deliberate operator nudges for busy-band edge cases where the auto-engine falls a slot behind or you can see the exchange is further along than the state machine knows. The auto-engine handles the normal case; these exist for when a quick nudge is faster than waiting through the 4-slot retry cycle.

TX power / SWR readout

After each burst the Tab5 queries `PC;`/`SW;` for instantaneous forward power and SWR and shows the result briefly in the status line: `Last TX: X.XW SWRx.xx [Ns]`. If SWR > 4.0 (indicating the QMX SWR-protection latch tripped), the firmware automatically cycles `TX;` / 150 ms / `RX;` to clear the latch so the radio is ready for the next TX slot without operator intervention.

CQ message presets

Long-press the **Call CQ** button to open the preset editor. Three message slots with radio buttons — check the active one. A `+ <call> <grid>` quick-insert appends your identity. Standard CQ constructions (`CQ DX OZ1LAV JO65FR`, `CQ POTA …`) and ≤13-char free text both encode via the general ft8_lib encoder. Presets persist to NVS. The Call CQ button label shows the selected message; a short tap transmits it.

QSO logging (ADIF)

Every completed FT8 QSO — pounce or CQ-run — is automatically written to an ADIF v3.1.4 file at `/spiffs/qso.adi` on the Tab5's internal flash. The log survives reboots and re-flashes.

Download. The web UI bottom bar shows **"N QSOs ↓"** once the first QSO is logged. Clicking it downloads `qso.adi` for import into any logging software (WSJT-X, Log4OM, DXKeeper, …) or for LoTW/POTA.app, which the Tab5 can't upload to directly (see below).

Upload to QRZ Logbook / eQSL — directly from the Tab5. Two more buttons appear next to the ADIF download link once you've logged a QSO: **"QRZ ↑"** and **"eQSL ↑"**. Tap either the first time and it prompts for credentials (QRZ: API key from *Logbook Data → API Key* on qrz.com; eQSL: your username and password — eQSL has no API-key scheme) and saves them on the Tab5, not just in the browser. Every tap after that uploads everything logged since the last successful upload — no need to re-enter credentials, and no risk of duplicate uploads, since the Tab5 tracks how far it's gotten into the log. If a record is rejected (bad credentials, quota, etc.) the run stops and reports the reason rather than skipping past it; fix the cause and tap again to resume from where it left off. eQSL accepts a whole batch of QSOs per request; QRZ's API takes one at a time, so a big log takes one HTTP round trip per QSO.

LoTW and POTA.app have no equivalent built-in path — LoTW requires a certificate-based TQSL signing step, and POTA.app has no upload API at all (browser login or email only). Use the ADIF download above for both.

Fields in each record:

ADIF field	Content
CALL	Their callsign
FREQ	Dial frequency in MHz (3 d.p., e.g. `14.074`)
BAND	Amateur band (e.g. `20M`)
MODE	`FT8`
SUBMODE	`FT8` (required by LoTW TQSL and eQSL for digital mode credit)
RST_SENT	Our SNR estimate (e.g. `-07`; `599` if unavailable)
RST_RCVD	Signal report received
QSO_DATE	UTC date `YYYYMMDD`
TIME_ON	UTC time `HHMMSS`
MY_CALL	Your callsign (from Identity in the drawer)
MY_GRIDSQUARE	Your grid
GRIDSQUARE	Their grid (from the decoded FT8 message)

Clear. `GET /api/adif/clear` from the web UI wipes the file and resets the worked-call cache.

> Set your callsign and grid in the settings drawer → **Identity** before your first QSO so they appear correctly in logged records.

---

6. Time sync

The Tab5 needs accurate UTC for FT8 slot timing. Sources in priority order (highest first):

Source	When applied
SNTP (WiFi)	Always authoritative whenever WiFi is connected and has synced at least once — sets system clock, Tab5 RTC, and NVS
Tab5 RTC	RX8130CE supercap-backed (~30–40 h retention) — applied at boot before QMX or WiFi are available
QMX `TM;`	Offline fallback only — applied when WiFi is down or has never synced (field/POTA with no WiFi)
FT8 signal timing	Sub-second correction from the decoded signal population's average timing, auto-applied continuously every slot while FT8 is decoding (damped, so a noisy single slot can't yank the clock) — deliberately tracks who you're actually trying to work, not absolute GPS/NTP truth. A manual one-shot version is also available via the **Sync Time** modal (see below)
Manual	Full HH:MM override in the **Sync Time** modal — for rare POTA sessions with neither WiFi nor QMX clock

Each accepted sync writes through to the RX8130CE so the clock persists across power-off. The Tab5 also polls `TM;` every 5 minutes in the background to catch QMX GPS lock events.

For POTA / no-WiFi use: uncheck **WiFi initiated** in the settings drawer. Time comes from the QMX on USB connect (or the supercap RTC if the Tab5 was recently synced). FT8 slot timing will be accurate from either source.

FT8 time calibration modal

On the FT8 screen, tap **Filter** → **Sync Time** to open the time calibration modal. It shows three large boxes: **[HH] : [MM] : [SS]**.

- **HH / MM** — pre-filled from the current UTC clock. Tap either box to edit it with a numpad. Use this for rare POTA situations where neither WiFi nor the QMX clock is available.

- **SS** — auto-syncs continuously from decoded FT8 signals. Each slot, every successfully decoded station contributes a timing sample; a robust outlier-rejecting average (median ± a tolerance window) across all of them sets the correction, so one bad decode can't throw off the reading. The box border flashes bright blue for ~30 ms each time a fresh measurement lands — a visual heartbeat that sync is active — and the hint below reads **"Flash: FT8 synced..."**. A blue frame means it is actively tracking; tap SS to lock it.
- **Apply** — if only SS was synced (HH and MM untouched), `time_sync_apply_correction_ms()` nudges the system clock by the measured sub-second offset and writes through to the RTC. If HH or MM was edited, `time_sync_set_manual()` sets the full time. The bottom bar shows **UTC(FT8)** after an FT8-derived correction, or **UTC(manual)** after a full manual set.

---

7. Reference

Gestures

Gesture	Where	Effect
Tap	Spectrum / waterfall	Tune to tapped frequency (snapped)
Touch + drag (hold ~250 ms first)	Spectrum / waterfall	Cyan cursor snaps grid-to-grid; tunes on release
Fast one-finger horizontal swipe	Spectrum / waterfall, any zoom	Pan/"stroll" the view; retunes to the new centre on release
Double-tap	Spectrum / waterfall	Reset zoom and pan to ×1.0 / centred
Pinch (two fingers)	Spectrum / waterfall	Zoom ×1.0–×24.0
Two-finger drag	Spectrum / waterfall (zoomed)	Pan the zoomed window
Swipe → from left edge	Left edge strip	Toggle Panadapter ↔ FT8 screen
Swipe ← from right edge	Right edge strip	Open settings drawer
Tap right grip handle	Right edge	Open settings drawer (alternative)
Swipe →	Open drawer, or spectrum while drawer open	Close settings drawer
Swipe ↑ from bottom edge	Bottom edge strip	Open memory channel picker
Tap or swipe ↓	Top-bar item (Band/Mode/BW/Freq/Zoom)	Open that item's selector

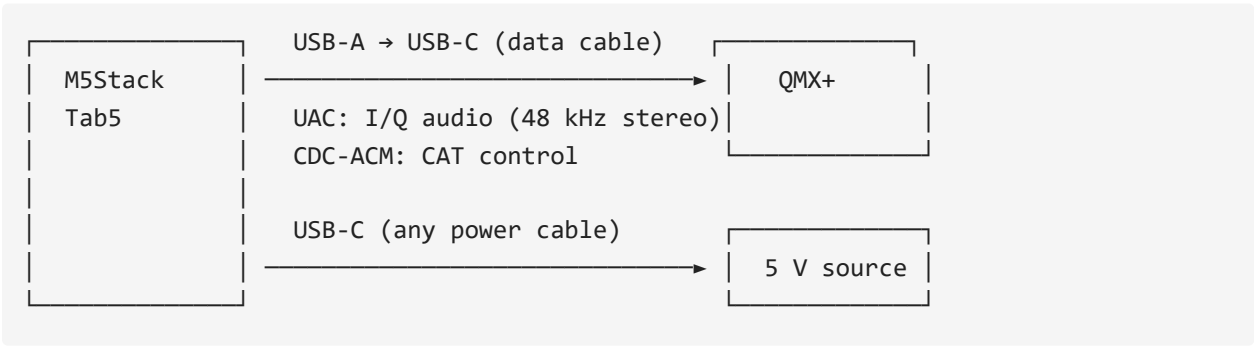
Gesture	Where	Effect
Touch and hold ~250 ms	FT8 decode list row	Dim preview at ~80 ms; full selection at 250 ms (scroll locks, drag moves highlight, lift to confirm)
Quick swipe	FT8 decode list	Scroll the list normally

Per-unit IF calibration

The QMX's +12 kHz IF injection varies slightly between units. If signals appear consistently shifted left or right of where the QMX is actually tuned, open the settings drawer → **IF calibration** slider (±100 Hz, persisted to NVS). Default 0. Reported by Ken KF0AYY, whose unit needed about −55 Hz to centre correctly.

Hardware

- **M5Stack Tab5** — ESP32-P4 v1.3 (ECO2), ST7121 or ST7123 5" 720×1280 MIPI-DSI touch display, 32 MB PSRAM, ESP32-C6 co-processor for WiFi
- **QRP Labs QMX or QMX+** — firmware 1.03.002 or newer required
- **USB-A → USB-C data cable** between Tab5 USB-A host port and QMX (full data, not charge-only)
- **USB-C power supply** for the Tab5 (5 V / 2 A or better, or internal battery)



What works without the QMX connected

Works without QMX	Needs QMX connected
UI, settings drawer, gestures	Spectrum + waterfall (no I/Q = flat line)
WiFi, web UI, screenshots	Top bar Band / Mode / BW
Callsign / grid / WiFi credentials	Tap-to-tune, mode popup (CAT writes)
Brightness, colour map, dB range	S-meter, FT8 decode/TX, QMX firmware readout

If the spectrum is flat and the top bar shows `---`, that is almost always the radio not being seen over USB — which loops back to the cable (or the QMX being off / in flash mode).

Reporting hardware issues

Near the top of the boot log you will see:

```
I (xxxx) bsp_info: === TAB5 BSP INFO ===
I (xxxx) bsp_info: chip:     ESP32-P4 rev v1.3
I (xxxx) bsp_info: psram:    30 MB
I (xxxx) bsp_info: panel:    ST7123 (inferred from touch)
I (xxxx) bsp_info: touch:    ST7123 @ 0x55
I (xxxx) bsp_info: heap:     230.5 kB internal free, 28.80 MB PSRAM free
I (xxxx) bsp_info: idf:      v5.4.4
I (xxxx) bsp_info: firmware: v0.18.7
I (xxxx) bsp_info: ====================
```

When opening a hardware issue, paste this block — the `panel` and `touch` lines are the first thing needed to identify which Tab5 hardware revision you have.

> **Note on `reset_reason`:** a deliberate reset-button force-off returns `panic/exception`, not `external-pin/power-on`. A genuine firmware crash always prints a `Guru Meditation` register + backtrace dump immediately *before* the reboot. No backtrace = abrupt power-off, not a crash.

Appendix — Technical Reference

The sections below are the remaining, more technical parts of the same project README (build instructions, internal design notes, roadmap) — included here so the links above resolve, rather than dead-ending. Skip ahead if you only want operating instructions.

Appendix A — Build from source

Requires **ESP-IDF v5.4.4** — pinned; do not upgrade. ESP32-P4 v1.3 silicon requires `CONFIG_ESP32P4_REV_MIN_0=y` and CPU capped at 360 MHz, both baked into `sdkconfig`.

```
# Activate the IDF environment, then:
idf.py build flash monitor
```

Or with the `qmx` PowerShell helper — add to your `$PROFILE` and adjust the COM port:

```
function qmx {
    param([string]$cmd = "fm")
    if (-not $env:IDF_PATH) { & C:\esp\v5.4.4\esp-idf\export.ps1 }
    switch ($cmd) {
        "b"   { idf.py build }
        "f"   { idf.py flash }
        "m"   { python -m esp_idf_monitor -p COM3 build\qmx_panadapter.elf }
        "fm"  { idf.py flash; python -m esp_idf_monitor -p COM3
         build\qmx_panadapter.elf }
        "bfm" { idf.py build flash; python -m esp_idf_monitor -p COM3
         build\qmx_panadapter.elf }
    }
}
```

Exit monitor: `Ctrl+T` then `Ctrl+X`.

Appendix B — Under the hood

I/Q balance correction

The QMX presents I/Q audio with a small but measurable amplitude imbalance and quadrature error between channels. Without correction, every signal produces a mirror image across the centre frequency (~30 dB weaker), cluttering the waterfall on a busy band.

A blind adaptive Gram-Schmidt orthogonaliser runs sample-by-sample in the audio pipeline before the FFT. It maintains running estimates of:

- **DC offset** on each channel (τ = 1 s)
- **Power** in I and Q (τ = 200 ms)
- **Cross-product** I·Q (τ = 1 s) — non-zero cross-product means the channels are not orthogonal

Correction per sample:

```
i_out = i
q_out = (q − K_phi · i) × K_amp
```

No calibration step needed; the estimator converges on ambient band noise within a few seconds of any signal. A two-speed startup runs all alphas at 8× for the first 2 s of real signal after each reset (effective convergence ~125 ms), then drops to steady-state. Toggle in the settings drawer; re-enabling resets the estimator so it reconverges from a clean state.

DSP pipeline

- **FFT:** 1024-pt complex, Blackman-Harris window. esp-dsp ANSI fallback is used (the PIE/vector version crashes under sustained WebSocket load on this silicon).
- **IF offset:** The QMX presents I/Q at +12 kHz; spectrum, waterfall, and S-meter all shift bin selection by `n_bins/4` so the VFO signal appears at the visual centre.
- **DC blocker:** one-pole IIR on the I/Q stream before FFT.
- **Spectrum smoothing:** per-bin EMA (α = 0.4 default, adjustable).
- **dBm calibration:** `DSP_DB_CALIBRATION_OFFSET = -148.0 dB` measured on dummy load; noise floor reads −130 dBm; S9 = −73 dBm.
- **Waterfall scroll:** 1280×824 double-height canvas (~130 µs/tick vs ~92 ms with memmove).

- **Audio task:** polling on core 0 with a drain loop. Event-driven reads caused noise-floor pumping (~13 s cycle) from truncated UAC chunks; this architecture eliminates it.

Quirks and trade-offs

PI4IO expander must be initialised before display bring-up. The Tab5's PI4IO I/O expander holds `LCD_RST` and `TP_RST` low at chip power-on. Without explicit init, on a true cold boot the panel never comes out of reset and the DSI FIFO hangs. Soft resets mask this because the expander retains state across ESP32 resets. `display_init` calls `bsp_i2c_init()` + `bsp_io_expander_pi4ioe_init()` first, then waits 120 ms.

Patched components in `components/`. `espressif__usb_host_uac/` has `create_background_task = true` — required for UAC and CDC-ACM to coexist on the same USB host. `espressif__esp_lcd_touch_st7123/` makes `FW_VERSION_REG` the only mandatory register read (ST7121 NACKs the others and also adds a `max_touches > 10` bounds clamp). Do not replace either with the registry version without re-applying the patches.

LVGL software rotation (~50% FPS cost). The panel is natively portrait; landscape is achieved via `lv_display_set_rotation(LV_DISPLAY_ROTATION_90)`. Every flush goes through `rotate90_rgb565`. ~13 fps landscape vs ~22 fps portrait — acceptable for a panadapter. PPA hardware rotation conflicts with the USB host stack over DW-GDMA channels and silently kills UAC + CDC-ACM; do not set `CONFIG_LVGL_PORT_ENABLE_PPA=y`. A full native-portrait rewrite would recover the lost FPS but isn't planned — accepted as a permanent trade-off.

IDLE watchdog disabled. The LVGL rotation pipeline keeps CPU0 busy past the default watchdog window. `CONFIG_ESP_TASK_WDT_CHECK_IDLE_TASK_CPU0/CPU1` are off; the app-task watchdog (30 s) is still active.

Cross-thread CAT writes must go through the poll task. Writing CAT commands directly from the LVGL thread races the FA/MD/FW poll on the same CDC pipe — commands interleave and the QMX returns `?;`. Pattern: stash the request in a `volatile` and let `poll_task` drain it on its next cycle. See `s_pending_ssb_bw` / `cat_request_ssb_bandwidth()` in `cat.c`.

SSB filter bandwidth needs three coordinated writes. `MMSSB|Filter RX=<hz>;` commits the value (persists, shows in QMX menu); `MMSSB|Bandwidth=<hz>;` applies it live; `FW;` polling must be suspended while the width is pinned because reading the filter makes the QMX revert it. Dead ends: `FW<nnnn>;` CAT set returns `?;`; re-asserting the same mode digit does not reload the filter; `Bandwidth` write alone reverts on the next poll.

HTTPS needs mbedtls allocating from PSRAM, not internal RAM. The QRZ/eQSL upload features (v0.16.2) are this firmware's first-ever outbound HTTPS connections — SNTP, the only prior network client, is UDP. With the default `CONFIG_MBEDTLS_MEM_ALLOC_MODE=MBEDTLS_INTERNAL_MEM_ALLOC`, the TLS handshake's 16 KB+ buffers competed for the ~200 KB internal DRAM already under pressure from USB host/audio/FFT/LVGL and every connection failed with `ESP_ERR_HTTP_CONNECT`. Fixed by switching to `MBEDTLS_EXTERNAL_MEM_ALLOC` in `sdkconfig`, which moves those buffers into the 28 MB of free PSRAM instead.

Project layout

```
main/
  main.c                    app_main, task launch, orchestration
  display/display.c         BSP bring-up (PI4IO + LCD + touch)
  ui/ui.c                   LVGL widgets, touch handler, canvases
  ui/ft8_screen_view.c      FT8 decode list, touch-drag row selection
  ui/ft8_tx_modal.c         TX confirmation modal
  cat/cat.c                 USB CDC-ACM + Kenwood CAT
  audio/audio.c             USB UAC + ring buffer producer
  dsp/dsp.c                 FFT, spectrum mutex, DC blocker
  dsp/iq_balance.c          Gram-Schmidt I/Q correction
  ft8_tx.c                  FT8 TX engine (build/arm/run/abort)
  ft8_test.c                FT8 slot loop (RX decode / TX burst)
  ft8_qso.c                 Auto QSO state machine
  ft8_status.c              Mutex-protected FT8 status string
  adif/adif_log.c           ADIF QSO logging
  adif/qrz_upload.c         QRZ Logbook API upload (one record per request)
  adif/eqsl_upload.c        eQSL.cc upload (batched, multiple records per request)
  rtc/rtc.c                 RX8130CE supercap RTC driver
  time_sync/time_sync.c     Time sync orchestrator
  render/render.c           30 Hz render task, EMA smoothing
  render/render_waterfall.c Waterfall scroll (double-height canvas)
  screenshot/screenshot.c   RGB565 framebuffer capture for /ss.bmp
  storage/settings.c        NVS-backed settings with debounced flush
  wifi/wifi.c               C6 co-processor WiFi + SNTP
  net/webserver.c           HTTP server + WebSocket
  util/fps.c                FPS counter
  util/diag_log.c           Diagnostic log ring buffer (vprintf hook)
```

Appendix C — Roadmap

The full per-version changelog — every release from v0.1.0 onward — lives in **docs/version-history.md**. This section tracks only what's planned next.

Next up

The path to v1.0 is a complete standalone FT8 station with TX, logging, and ADIF upload.

- **LoTW upload.** Needs its own design — certificate-based via TQSL, not a simple HTTP API like QRZ/eQSL.
- **v1.0.0 — Stable release.** Multi-day FT8 soak complete, polished UI, beta label gone.

Longer term

- **CW decoder.** Goertzel-based, text scrolling under the spectrum. The QMX already does this internally — question is whether to mirror its output via CAT or run a parallel decoder on the Tab5.
- **Tab5 speaker / headphone audio.** Demodulated CW/SSB passband audio out of the Tab5's own jack, so the operator can monitor without the QMX's audio path.
- **Extended waterfall history.** PSRAM has room for several minutes of scrollback; two-finger drag to scrub through history.
- **QMX (small) support.** Same UI, different USB endpoint config and band table.
- **JS8 / RTTY modes.** See `docs/js8-feasibility.md` and `docs/rtty-feasibility.md`.
- **FT4 mode** (requested by Roy). Lower-effort than JS8/RTTY: the vendored `ft8_lib` already fully parameterizes FT4 vs FT8 internally (`FTX_PROTOCOL_FT4` — Costas pattern, 4-tone Gray map, symbol period, LDPC(174,91) all already implemented, see `components/ft8_lib/ft8/constants.h` and the `wf->protocol == FTX_PROTOCOL_FT4` branches in `decode.c`), so decode/encode itself is essentially free. The app-level work is the slot/timing plumbing built around the FT8 assumption of a fixed 15 s slot: `ft8_test.c`'s capture/decode/TX slot loop, `ft8_qso.c`'s timeout-in-slots counters, the UI countdown bar, and CAT DigiMode forcing all need a parallel 7.5 s FT4 path (or a mode-aware slot-length parameter threaded through). Rough order of magnitude: a fraction of the RTTY/JS8 estimates in the feasibility docs above — no new DSP pipeline, no new UI screen, mainly a mode switch threaded through existing FT8 plumbing.
- **DSP polish.** Noise reduction, auto-notch.

Appendix D — Related projects

- DX-FT8 by Barb (WB2CBA) — open-hardware FT8 tablet transceiver; an inspiring reference for a similar use-case
- `qrp_companion` by Zhenxing Han (N6HAN) — Tab5 companion for QMX with audio + CAT; source of the polling audio task pattern and battery readout approach
- `ft8_lib` by Karlis Goba — FT8 encoder/decoder vendored as `components/ft8_lib`

Appendix E — License